

Python Practice Handout

For loops

1. Your first for loop

A for loop runs the same piece of code over and over. Write for, a variable name, in, then range with a number, then a colon. The line underneath is tabbed across, just like an if statement.

```
for number in range(10):  
    print("Hello")
```

Output (first four lines):

```
Hello  
Hello  
Hello  
Hello
```

In your console you get ten Hellos, one for each pass. range(10) means go round the loop ten times.

2. Every tabbed line repeats

Anything tabbed under the for line runs on every pass. Here both lines repeat, so you get twenty lines in total.

```
for number in range(10):  
    print("Hello")  
    print("Hi there")
```

Output (first four lines):

```
Hello  
Hi there  
Hello  
Hi there
```

3. What the variable holds

The variable after for holds a different value each time round. Print it and you can watch it change. Counting starts at zero, exactly like list indexes.

```
for number in range(10):  
    print(number)
```

Output:

```
0
1
2
3
4
5
6
7
8
9
```

Notice it stops at 9, not 10. Ten numbers starting from zero.

4. The name is up to you

That variable can be called anything. Programmers often use x. When you are looping through numbers, number is a clearer name.

```
for x in range(10):
    print(x)
```

This gives exactly the same output as before.

5. Looping through a list

Lists and for loops work beautifully together. Instead of range, give it a list, and the variable holds each item in turn.

```
fav_movies = ["Sandlot", "The Lego Movie", "Dune"]

for movie in fav_movies:
    print(movie)
```

Output:

```
Sandlot
The Lego Movie
Dune
```

6. Loops are fast

Make the range as big as you like. A thousand numbers takes about a second. A hundred thousand takes a few seconds. Writing those out by hand would take you weeks.

```
for number in range(100000):
    print(number)
```

Be careful with very large ranges. Printing that many lines can slow the console right down.

7. Try it yourself

Loop 40 times and print the number of the loop multiplied by two, so it goes 2, 4, 6, 8 and on up. Try this on your own before looking below.

Start by printing the plain number to check the loop works.

```
for number in range(40):  
    print(number)
```

Multiplying by two gets close, but because counting starts at zero the first answer is 0 rather than 2.

```
for number in range(40):  
    print(number * 2)
```

Output (first four lines):

```
0  
2  
4  
6
```

8. Brackets change the order

Adding one fixes the starting point, but Python does multiplication before addition, so `number * 2 + 1` is not what you want. Put brackets around the part that must happen first.

```
for number in range(40):  
    print((number + 1) * 2)
```

Output (first four lines, ending at 80):

```
2  
4  
6  
8
```

Without the brackets you would get 1, 3, 5, 7 instead. The brackets say add one first, then double it.